Technical Documentation

Project: RAG-Based Customer Support Assistant

1. Introduction

- **What is RAG?** Retrieval-Augmented Generation (RAG) is an AI framework that grounds Large Language Models (LLMs) on external, verifiable knowledge bases. Instead of relying solely on the model's pre-trained weights, RAG retrieves relevant facts from a database and provides them to the LLM to generate an informed answer.
- **Why it is needed:** LLMs are prone to "hallucinations"—generating plausible but factually incorrect information. In enterprise environments, accuracy is paramount. RAG solves this by restricting the AI's answers to specific, trusted documents.
- **Use Case Overview:** This system acts as a Tier 1 Customer Support Agent. It reads a company's policy PDF, answers routine customer questions accurately, and intelligently knows when to step aside and let a human handle complex edge cases.

2. System Architecture Explanation

The architecture relies on a decoupled design. The **Knowledge Ingestion Pipeline** operates independently to process and vectorize the PDF into ChromaDB. The **Inference Pipeline** is orchestrated by LangGraph. When a query enters, it does not go straight to the LLM. It first goes to the vector database to fetch factual context. The system then enters a unique "Intent Evaluation" phase where the LLM acts as a judge, reviewing the retrieved facts against the query. Based on this judgment, the system either generates an automated response or triggers the Human-in-the-Loop escalation path.

3. Design Decisions

- **Chunk Size Choice:** A chunk size of 1000 characters with an overlap of 200 characters ensures that paragraphs are generally kept intact, preserving context, while the overlap prevents sentences from being cut in half across different chunks.
- **Embedding Strategy:** Dense vector embeddings capture the semantic meaning of sentences, allowing the system to match queries like "How do I fix my screen?" with document text like "Display troubleshooting," even if exact keywords aren't used.
- **Retrieval Approach:** We use top-k retrieval (k=3 or 4) to provide the LLM with enough context to form a complete answer without overflowing the context window or diluting the prompt with irrelevant information.
- **Prompt Design Logic:** Strict system prompts are utilized. The intent prompt explicitly forces a binary "yes/no" output for easy routing. The generation prompt explicitly commands: *"Answer strictly based on the provided context. If the answer is not present, say so."*

4. Workflow Explanation

- **LangGraph Usage:** LangGraph is used to build a stateful, multi-actor application. It models the workflow as a graph consisting of nodes and edges.
- **Node Responsibilities:** * Retrieve: Interfaces with ChromaDB.
    - Evaluate: Interfaces with the LLM for binary decision making.

    - Generate: Interfaces with the LLM for natural language generation.
    - HITL: Interfaces with standard input for human data entry.
- **State Transitions:** As the process moves from node to node, it updates a central GraphState dictionary. For example, the Retrieve node finds text and updates state['context'], which the Evaluate node then reads.

5. Conditional Logic

- **Intent Detection:** This is the core safeguard of the system. By asking the LLM to rate its own ability to answer the question using *only* the retrieved context, we create a deterministic checkpoint in an otherwise non-deterministic AI process.
- **Routing Decisions:** The routing function looks at the intent state. If the string contains "no", the edge dynamically connects to the HITL node. Otherwise, it connects to the Generate node.

6. HITL Implementation

- **Role of Human Intervention:** The HITL system acts as the ultimate safety net. It ensures that no customer is left with a hallucinatory answer or a dead-end "I don't know" response.
- **Benefits and Limitations:** * *Benefits:* Guarantees 100% accuracy on complex queries; builds customer trust; captures edge cases for future system improvements.
    - *Limitations:* Introduces asynchronous delays (the bot pauses until the human replies); requires staff availability.

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed:** Retrieving more chunks (higher *k*) improves the chances of finding the right answer but increases the token count, slowing down the LLM generation time and increasing API costs.
- **Chunk Size vs. Context Quality:** Smaller chunks yield highly precise vector matches but might lack the surrounding sentences needed for the LLM to understand the full picture.
- **Cost vs. Performance:** Using advanced models like GPT-4o for the "Evaluate" node yields highly accurate routing but is expensive. Using a smaller model reduces costs but might lead to false escalations.

8. Testing Strategy

- **Testing Approach:** The system should be tested using a matrix of expected vs. unexpected queries to observe routing behavior.
- **Sample Queries:**
    - *Standard:* "What is the warranty period for this product?" (Expected: Automatic RAG answer).
    - *Out of Scope:* "What is the capital of Japan?" (Expected: Fails retrieval/intent -> Routes to HITL).

    - *Complex:* "If I bought this two years ago and dropped it in water, but I have the extended plan, am I covered?" (Expected: LLM recognizes conflicting context -> Routes to HITL).

9. Future Enhancements

- **Multi-document Support:** Upgrading the document loader to ingest entire directories of PDFs, URLs, and text files to build a comprehensive knowledge base.
- **Feedback Loop:** Adding a "Thumbs Up/Down" feature to the UI. Downvotes can flag the query/response pair to fine-tune the retrieval parameters or update the base PDF.
- **Memory Integration:** Adding ConversationBufferMemory to the graph state so the bot can answer follow-up questions (e.g., User: "What is the policy?" -> Bot: "It is X." -> User: "Does that apply to me?").
- **Deployment:** Transitioning the CLI application into a scalable web app using FastAPI for the backend endpoints and Streamlit or React for the frontend interface.